Year: 2026-2027 **Sem V - NEP**

Course: Agile Software Development & DevOps Lab

Class: T1 **Date of Submission: 15/10/2026**

A Mini Project Report on

CampusFix

An Automated DevOps Campus Grievance Platform Delivered Through an Agile and CI/CD Pipeline

Submitted in partial fulfilment of the requirements of the course

Agile Software Development & DevOps Lab

Semester V (NEP) · Academic Year 2026–2027 · Class **T1**

Submitted by

Name	Roll No.
Kripa Masand	2401061
Moheet Dialani	2301028
Ronak Duseja	2301030

Under the guidance of

Prof. Sanober Shaikh

Department of **Artificial Intelligence and Data Science**,
Thadomal Shahani College of Engineering

2026–2027

CERTIFICATE

This is to certify that the mini project entitled **“CampusFix: An Automated DevOps Campus Grievance Platform Delivered Through an Agile and CI/CD Pipeline”** is a bona fide work of

Kripa Masand 2401061, **Moheet Dialani 2301028** and **Ronak Duseja 2301030** submitted in partial fulfilment of the requirements of the course Agile Software Development & DevOps Lab, Semester V, during the academic year 2026–2027.

Prof. Sanober Shaikh	**Dr. Madhuri Rao**
Project Guide	Head of Department

Date: _____ Place: _____

ABSTRACT

Educational institutions still handle everyday facility complaints—a leaking tap in a hostel, a dead projector in a lecture hall, unreliable Wi-Fi in a computer lab—through verbal requests, scattered messages and e-mail threads. Complaints get lost, nobody can see their status, and administrators have no consolidated view of what is pending. **CampusFix** addresses this by providing a centralised, authenticated grievance portal in which students file categorised complaints (Hostel, Lab, Classroom, Mess, General) and administrators move each complaint through a transparent lifecycle: **Open → In Progress → Resolved**.

CampusFix was developed as the core artifact of the mini-project for **Agile Software Development & DevOps Lab**, and its main purpose is to show how the seven laboratory experiments combine into a single, working delivery pipeline. The application is a **Flask 3** web service served by **Gunicorn**, backed by a **PostgreSQL 15** database through Flask-SQLAlchemy. Passwords are stored as salted PBKDF2-SHA256 hashes (Werkzeug), the registration route always assigns the student role to prevent privilege escalation, and the service exposes `/health` (database liveness probe), `/metrics` (Prometheus exporter) and `/api/stats` (JSON summary) endpoints for operations.

The delivery lifecycle is fully automated. Source code is version-controlled in **Git/GitHub**; a **multi-stage Dockerfile** (`builder`, `tester`, `runner`) produces a slim, non-root production image with a native `HEALTHCHECK`; **Docker Compose** orchestrates the web application, database, node-exporter, Prometheus and Grafana on an isolated bridge network with persistent volumes and health-gated start-up; and a declarative five-stage **Jenkinsfile** checks out the code, runs an **11-case pytest suite** inside the test container, publishes the JUnit report, builds the production image, deploys the stack and performs an automated `/health` smoke test. Delivery was planned in **Jira** (five epics, eight user stories, two sprints), and runtime behaviour is observed with **Prometheus and Grafana** dashboards showing service health, request rate, p95 latency, complaint counts and host CPU, memory and disk utilisation.

All 11 automated tests pass (11 passed, 0 failed, 0 errors), and the complete stack starts from a single `docker compose up -d --build` command.

Keywords: Agile, Scrum, Jira, Git, Flask, PostgreSQL, Docker, Docker Compose, Jenkins, CI/CD, pytest, Prometheus, Grafana

CONTENTS

Chapter	Title
1	Introduction
2	Requirements and Agile Planning (Experiment 6)
3	System Design
4	Application Implementation
5	DevOps Implementation (Experiments 1–7)
6	Testing
7	Results and Discussion
8	Conclusion and Future Scope
	References
A	Appendix A: Running the Project
B	Appendix B: Team Contributions

1. Introduction

1.1 Background

A college campus runs on dozens of small facilities—hostel rooms, laboratories, classrooms and mess halls—and each of them generates a steady stream of faults and requests. In most institutions these are reported verbally, on paper registers or through informal messaging groups. The reporter rarely learns whether the issue was seen, who owns it, or when it will be fixed, and the administration cannot easily identify recurring problems or measure how quickly they are resolved.

A web-based grievance system solves this by giving every complaint a record, an owner-visible status and a timestamp. Building such a system is also an ideal vehicle for modern engineering practice, because it is small enough to finish in a semester yet realistic enough to need authentication, a relational database, automated testing, containerised deployment and monitoring.

The Agile Software Development & DevOps laboratory introduced the practices needed to build and deliver software reliably: distributed version control (Git), containerisation (Docker), multi-container orchestration (Docker Compose), continuous integration and delivery (Jenkins), the Agile project lifecycle (Jira) and observability (Prometheus and Grafana). Each experiment was originally performed on a small, isolated workload. This mini-project brings them together in **CampusFix**, so that their interaction and the full code-to-production lifecycle can be demonstrated on one cohesive product.

1.2 Problem Statement

To design, develop, containerise and operate a full-stack campus grievance management web application (**CampusFix**) that lets students register, log in and submit categorised complaints, lets administrators review all complaints and update their status through a defined lifecycle, and exposes health and metric endpoints for operations; and to deliver the entire system through an Agile, automated DevOps lifecycle in which changes are version-controlled in Git, verified by an automated test suite inside a declarative Jenkins pipeline, packaged as a hardened multi-stage Docker image, deployed with Docker Compose, smoke-tested automatically and continuously observed with Prometheus and Grafana.

1.3 Objectives

1. **User Authentication & Access Control:** Provide registration and login with salted password hashing and role-based sessions (student and admin), and prevent students from escalating privileges through the registration form.
2. **Grievance Submission:** Allow authenticated students to submit complaints with a title, category (Hostel, Lab, Classroom, Mess, General) and description, and to view their own complaints with status and timestamp.
3. **Administrative Resolution Workflow:** Give administrators a dashboard listing all complaints and the ability to move each one through Open, In Progress and Resolved.
4. **Containerisation (Experiment 2):** Package the application with a three-stage Dockerfile (`builder`, `tester`, `runner`) that runs as a non-root user and carries a built-in health check.
5. **Orchestration (Experiment 3):** Run the web application, PostgreSQL, node-exporter, Prometheus and Grafana together using Docker Compose with named volumes, an isolated network and health-based dependencies.
6. **Automated Testing & CI (Experiment 4):** Create a pytest suite of 11 unit and integration tests with isolated database fixtures that produces a JUnit XML report consumed by Jenkins.

7. **Automated Deployment & Smoke Testing (Experiment 5):** Build a declarative Jenkins pipeline that deploys the stack with Docker Compose and verifies the `/health` endpoint returns HTTP 200 before declaring success.
8. **Version Control & Rollback (Experiment 1):** Use a branch-and-pull-request workflow on GitHub and demonstrate recovery from a deliberately broken build with `git revert`.
9. **Agile Planning (Experiment 6):** Plan and track the work in Jira using epics, user stories with acceptance criteria and story points, issue links, two sprints and a burndown chart.
10. **Observability (Experiment 7):** Expose application metrics and visualise service health, request rate, latency, complaint volume and host resource usage in Grafana.

1.4 Scope

CampusFix targets a single campus and supports two roles—student and administrator—across five complaint categories. E-mail or SMS notifications, file attachments, complaint assignment to named staff members, and integration with a college single-sign-on service are outside the scope of this mini-project. The deployment target is a single host running Docker Engine; the same Compose file and Jenkins pipeline apply unchanged to a college lab machine or a cloud virtual machine.

1.5 Mapping of Lab Experiments to the Project

Exp.	Lab topic	Application in CampusFix	Section
1	Version control with Git	Feature branches, pull requests with merge commits, deliberate-failure and `git revert` rollback demonstration	5.1
2	Containerisation with Docker	Three-stage Dockerfile (builder, tester, runner), non-root `appuser`, `HEALTHCHECK`	5.2
3	Docker Compose	Five-service stack, named volumes, one bridge network, health-gated start-up	5.3
4	Jenkins CI pipeline	Declarative pipeline: checkout, test in container, JUnit report, image build	5.4
5	Automated deployment	Compose deployment from Jenkins and automated `/health` smoke test	5.5
6	Agile project lifecycle	Jira Scrum project CF: 5 epics, 8 stories, 2 sprints, burndown	2
7	Monitoring	Prometheus scraping the app and node-exporter; Grafana dashboard with 7 panels	5.6

Table 1.1: Lab experiments and where each is applied

2. Requirements and Agile Planning

2.1 User Roles

Role	Goal	Main screens / interfaces
Student	Register, log in, report a campus problem and track its status.	Home (`/`), Register (`/register`), Login (`/login`), My Complaints (`/complaints`)
Administrator	View every complaint and move it through Open, In Progress and Resolved.	Administrator Dashboard (`/admin`)
DevOps / Operator	Verify that the service is healthy, deploy changes and watch system behaviour.	`/health`, `/metrics`, `/api/stats`, Jenkins console, Prometheus (9090), Grafana (3000)

Table 2.1: User roles

2.2 Functional Requirements

- **User registration:** A visitor creates an account with a username and password. Empty fields are rejected, usernames are unique (compared case-insensitively) and the stored password is a salted hash.
- **Login and sessions:** Valid credentials start a session holding user id, username and role. Students are sent to their complaint page and administrators to the admin dashboard; invalid credentials show an error.
- **Complaint submission:** A logged-in student submits a title, category and description. The record is saved with status `Open`, the author and a UTC timestamp, and a Prometheus counter is incremented.
- **Complaint listing:** Students see only their own complaints, newest first, with a status badge and timestamp.
- **Admin resolution workflow:** The admin dashboard lists all complaints. The administrator can set a complaint to `Open`, `In Progress` or `Resolved`; any other value is ignored and a confirmation message is shown on success.
- **Access control:** Anonymous users are redirected to the login page; non-admin users cannot open `/admin`.
- **Operational endpoints:** `/health` reports service and database status (HTTP 200 or 500), `/metrics` exposes Prometheus metrics and `/api/stats` returns total, open, in-progress and resolved counts as JSON.

2.3 Non-Functional Requirements

- **Security:** Passwords are hashed with PBKDF2-SHA256; the registration route forces the `student` role; the container runs as non-root user `appuser` (UID 10001); configuration such as the database URL and secret key is read from environment variables.
- **Reliability:** The web container starts only after PostgreSQL reports healthy (`pg_isready`); containers use restart policies; database files persist in the named volume `postgres_data`.
- **Portability:** The complete stack starts with one command (`docker compose up -d --build`) on any machine with Docker Engine, over the isolated network `campusfix-network`.

- **Maintainability and testability:** A pinned `requirements.txt`, an 11-case automated test suite with isolated fixtures, and a JUnit report on every pipeline run.
- **Observability:** Request count, latency histogram, complaint counter and host metrics are scraped every 5 seconds and visualised in Grafana.
- **Performance:** Gunicorn serves the application with one worker and four threads, a deliberate choice that avoids database-initialisation race conditions between multiple worker processes while still handling concurrent requests.

2.4 Scrum Setup

The project followed the Scrum framework practised in the laboratory. A Jira Scrum project with key **CF** was created, and the work was organised into **five epics** (Table 2.2) and **eight user stories** (CF-10 to CF-17). User-facing capabilities were written in the form "As a <role>, I want <capability>, so that <benefit>" with explicit acceptance criteria and estimated in Fibonacci story points (3, 5, 8). Dependencies between stories were recorded as Jira issue links, for example "CF-16 is blocked by CF-15", because the pipeline cannot exist before the tests it runs.

Work was planned into **two sprints**: Sprint 1 delivered the application core (authentication, grievance submission and the admin workflow), and Sprint 2 delivered the DevOps layer (containers, Compose, automated tests, Jenkins pipeline and monitoring). Story progress was tracked on the sprint board (To Do, In Progress, Done) and summarised on the burndown chart.

Epic	Name	Description
CF-EPIC-1	User Authentication & Access Control	Student registration, session login, admin role enforcement
CF-EPIC-2	Complaint Management Workflow	Grievance creation, category classification, status transitions
CF-EPIC-3	DevOps Containerisation & Orchestration	Multi-stage Docker packaging, Docker Compose database networking
CF-EPIC-4	Automated CI/CD Pipeline	Jenkinsfile automation, pytest verification, automated deployment
CF-EPIC-5	Observability & Health Monitoring	Health endpoint, Prometheus scraping, Grafana dashboard

Table 2.2: Epics

2.5 Product Backlog

Key	Epic	User story / summary	Pts	Sprint
CF-10	EPIC-1	As a student, I want to register and log in so that I can submit grievances. Acceptance: hashed passwords, duplicate usernames rejected, session created on login.	3	1
CF-11	EPIC-2	As a student, I want to submit a complaint with title, category and description. Acceptance: saved with status Open.	5	1
CF-12	EPIC-2	As an administrator, I want to view all complaints and update their status (Open → In Progress → Resolved). Acceptance: DB updated and confirmation shown.	5	1

Key	Epic	User story / summary	Pts	Sprint
CF-13	EPIC-3	Package the Flask app with a multi-stage Docker build and non-root user. Blocked by CF-11.	5	2
CF-14	EPIC-3	Orchestrate app and PostgreSQL with Docker Compose, persistent storage and health checks. Acceptance: one command starts everything.	3	2
CF-15	EPIC-4	Automated pytest suite covering routes, authentication and health checks; JUnit XML for Jenkins. Acceptance: 11/11 tests pass.	3	2
CF-16	EPIC-4	Declarative Jenkinsfile that tests, builds and deploys containers. Acceptance: five stages, stops on test failure. Blocked by CF-15.	8	2
CF-17	EPIC-5	Expose `/metrics`, scrape with Prometheus every 5 s and build a live Grafana dashboard.	5	2

Table 2.3: Product backlog (total 37 story points: Sprint 1 = 13, Sprint 2 = 24)

2.6 Jira Artefacts

The following screenshots document the Agile lifecycle in Jira.

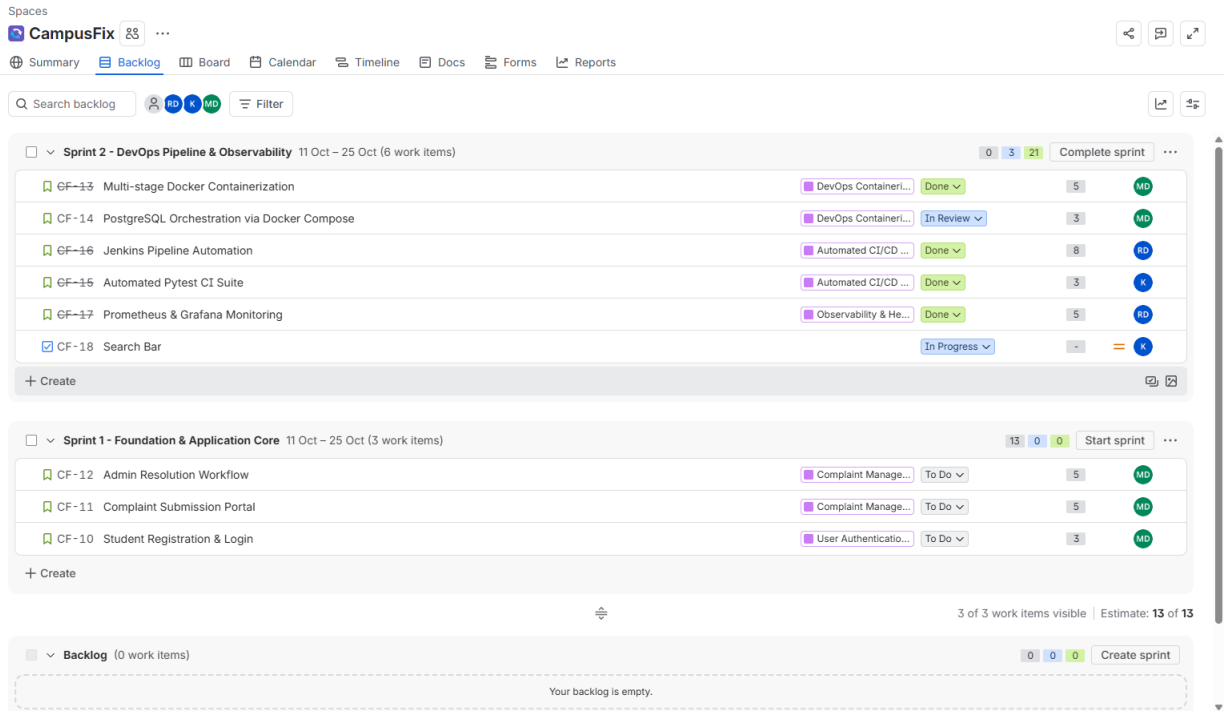


Figure 2.1: Jira backlog view showing Sprint 1 and Sprint 2 stories with their epics and story points

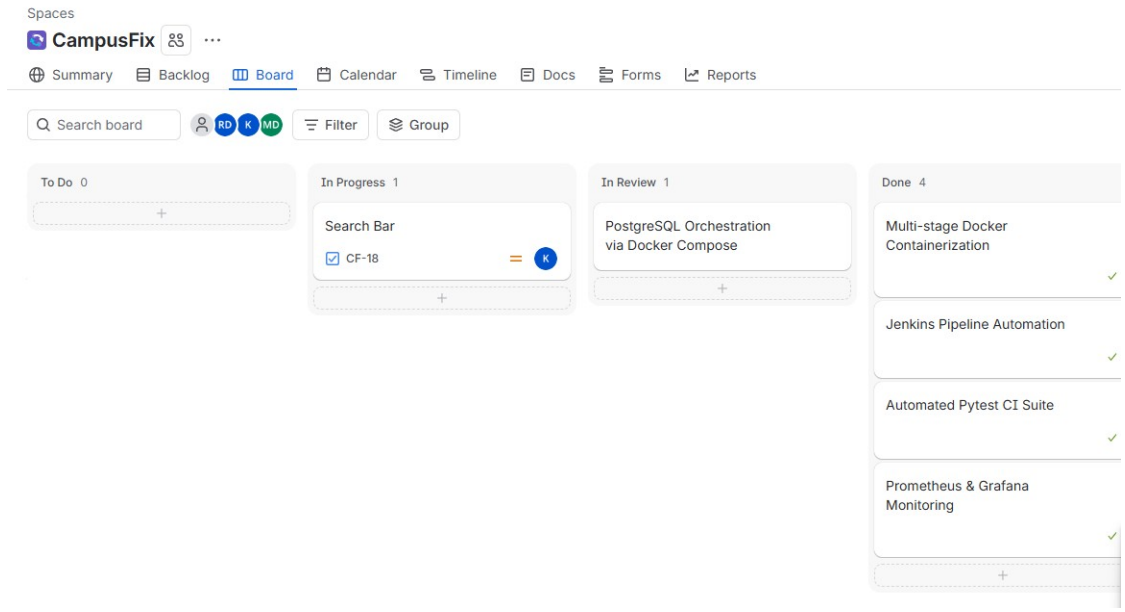


Figure 2.2: Active sprint board (Sprint 2) showing To Do, In Progress, In Review and Done columns

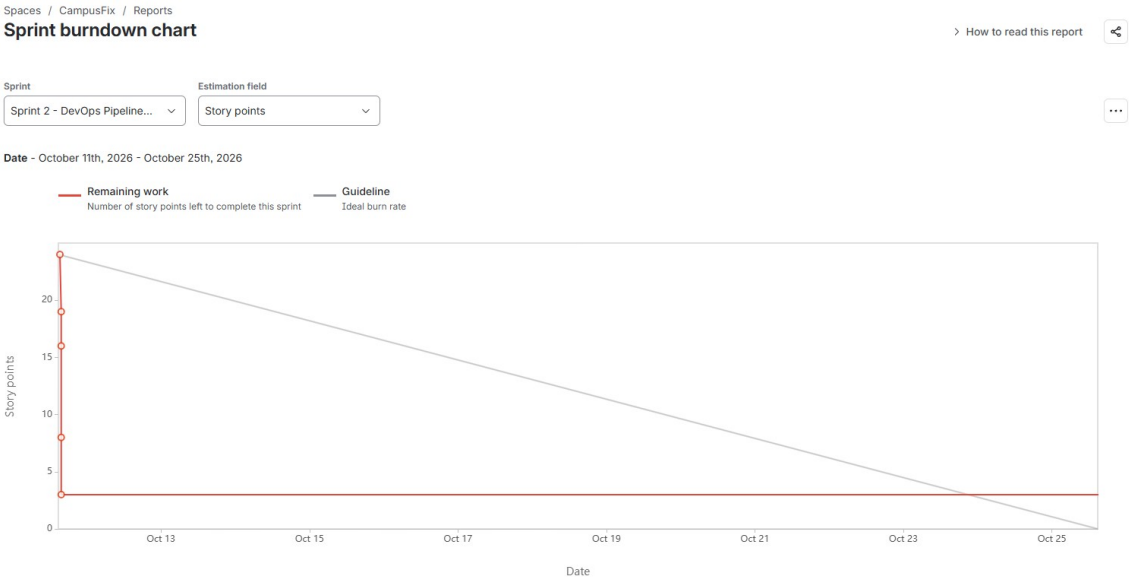


Figure 2.3: Sprint 2 burndown chart (11 Oct – 25 Oct 2026)

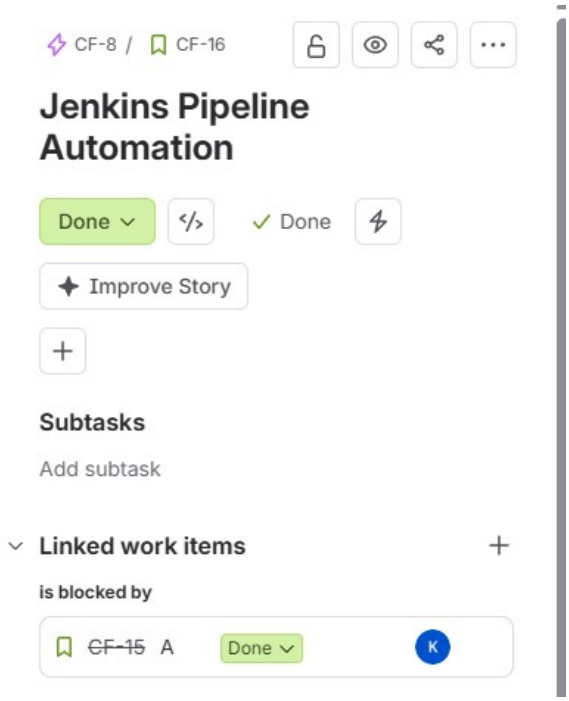


Figure 2.4: Issue detail page showing the link “CF-16 is blocked by CF-15”

3. System Design

3.1 Architecture

CampusFix is a containerised multi-service application orchestrated by Docker Compose, as shown in Figure 3.1. Its parts are:

1. **Application tier (web):** a Flask 3 application served by Gunicorn on port 5000. It renders server-side HTML templates (Bootstrap 5 based), handles authentication and sessions, implements the complaint workflow and exposes the operational endpoints `/health`, `/metrics` and `/api/stats`.
2. **Data tier (db):** PostgreSQL 15 (Alpine image) on port 5432 with the named volume `postgres_data`, so that users and complaints survive container restarts. Tables are created and default accounts seeded by the application on start-up.
3. **Monitoring tier:** `node-exporter` publishes host CPU, memory and disk metrics on port 9100; **Prometheus** scrapes it together with the application's `/metrics` endpoint every five seconds; **Grafana** on port 3000 visualises the data using a pre-built dashboard.
4. **Delivery tier (outside the stack):** GitHub holds the source and Jenkins runs the declarative pipeline that tests, builds and deploys the stack with Docker Compose.

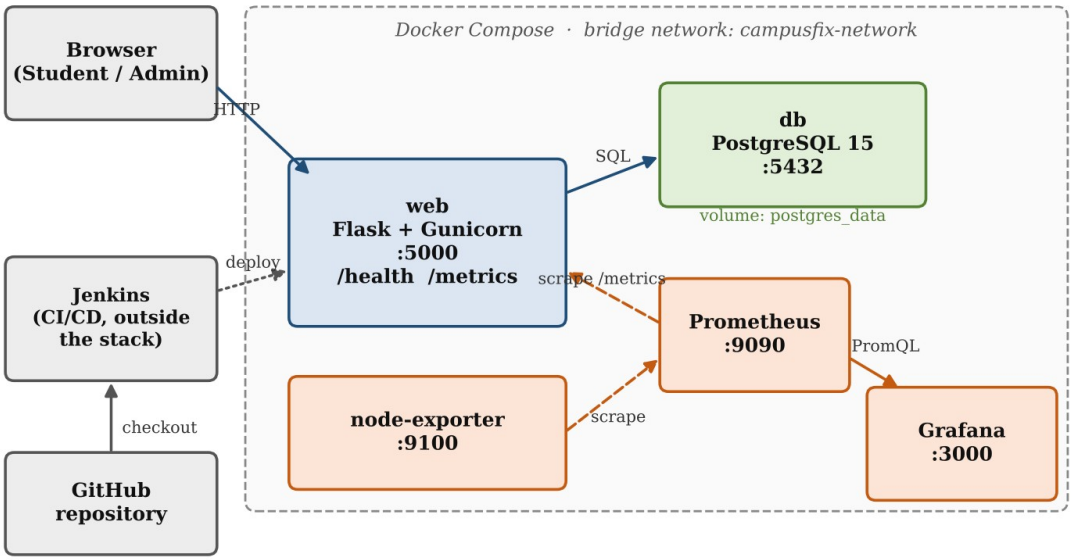


Figure 3.1: System and delivery architecture

3.2 Technology Stack

Layer	Technology	Reason for choice
Backend	Python 3.11, Flask 3.0.3	Lightweight framework; routes, sessions and templates with minimal boilerplate.
ORM / Database	Flask-SQLAlchemy 3.1.1, PostgreSQL 15 (SQLite fallback for local runs and tests)	Relational integrity between users and complaints; zero-setup SQLite when `DATABASE_URL` is not set.

Layer	Technology	Reason for choice
Security	Werkzeug PBKDF2-SHA256 hashing	Salted password hashes instead of plain text.
WSGI server	Gunicorn 23.0.0 (1 worker, 4 threads)	Production-grade server; single worker prevents duplicate database initialisation.
Frontend	Jinja2 templates, Bootstrap 5, custom CSS/JS	Server-rendered pages with no separate build step.
Testing	pytest 8.3.3, JUnit XML	Simple fixtures and a report format that Jenkins understands natively.
Containers	Docker multi-stage build, Docker Compose	Reproducible, small, non-root images; one-command orchestration.
CI/CD	Jenkins (declarative pipeline); GitHub Actions workflow as a secondary check	Automated test, build, deploy and smoke-test stages.
Monitoring	prometheus-flask-exporter 0.23.1, Prometheus, Grafana, node-exporter 1.8.2	Application and host metrics with live dashboards.
Agile	Jira Software Cloud (Scrum)	Epics, stories, sprints and burndown reporting.

Table 3.1: Technology stack

3.3 Database Design

The schema has two tables related one-to-many: one user can file many complaints. Passwords are stored as hashes in a 255-character column, and `created_at` is stored in UTC.

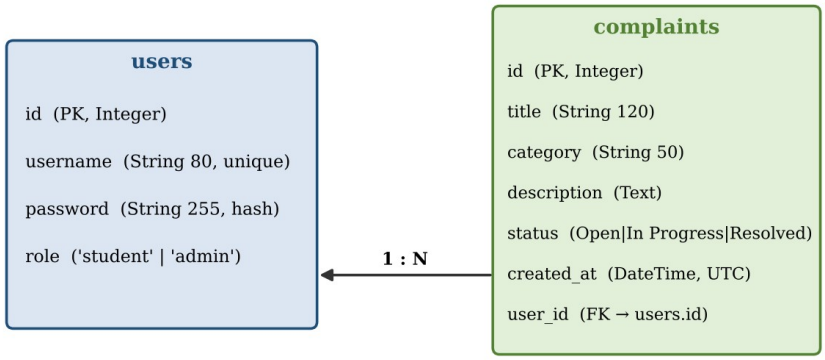


Figure 3.2: Entity relationship between users and complaints

Table	Column	Type	Constraint / default
users	id	Integer	Primary key
users	username	String(80)	Unique, not null
users	password	String(255)	Not null (hashed)
users	role	String(20)	Default `student`; `student` or `admin`
complaints	id	Integer	Primary key
complaints	title	String(120)	Not null
complaints	category	String(50)	Not null (Hostel, Lab, Classroom, Mess, General)

Table	Column	Type	Constraint / default
complaints	description	Text	Not null
complaints	status	String(20)	Default `Open`; Open, In Progress or Resolved
complaints	created_at	DateTime	Default current UTC time
complaints	user_id	Integer	Foreign key to users.id, not null

Table 3.2: Database schema

3.4 Complaint Lifecycle

A complaint is created with status **Open** when a student submits it. When an administrator starts work, the status is changed to **In Progress**, and when the issue is fixed it becomes **Resolved**. The admin route accepts only these three values, so a malformed request cannot put a complaint into an undefined state.

3.5 Routes and Endpoints

Method	Route	Access	Purpose
GET	/	Public	Landing page (redirects logged-in users to their complaints)
GET, POST	/register	Public	Create a student account
GET, POST	/login	Public	Authenticate and start a session
GET	/logout	Logged in	Clear the session
GET, POST	/complaints	Student	List own complaints / submit a new one
GET, POST	/admin	Admin only	List all complaints / update status
GET	/health	Public	Liveness probe with database check (200 / 500)
GET	/metrics	Public	Prometheus metrics
GET	/api/stats	Public	JSON summary of complaint counts by status

Table 3.3: Application routes

3.6 Repository Structure

```
campusfix/
├── app.py                    # Flask app: models, routes, health & metrics
├── Dockerfile                # 3-stage build: builder, tester, runner
├── docker-compose.yml        # web, db, node-exporter, prometheus, grafana
├── Jenkinsfile               # 5-stage declarative pipeline
├── requirements.txt          # pinned dependencies
├── pytest.ini
├── templates/                # base, index, login, register, complaints, admin
├── static/                   # style.css, main.js
├── tests/                    # conftest.py, test_app.py (11 tests)
├── monitoring/               # prometheus.yml, grafana-dashboard.json
└── .github/workflows/ci.yml  # GitHub Actions workflow
```

4. Application Implementation

4.1 Data Models

The two tables are mapped to SQLAlchemy models. The relationship lets a complaint reach its author through `complaint.author` and a user reach their complaints through `user.complaints`.

```
class User(db.Model):
    __tablename__ = "users"
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True, nullable=False)
    password = db.Column(db.String(255), nullable=False)
    role = db.Column(db.String(20), default="student")  # 'student' or 'admin'
    complaints = db.relationship("Complaint", backref="author", lazy=True)

class Complaint(db.Model):
    __tablename__ = "complaints"
    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String(120), nullable=False)
    category = db.Column(db.String(50), nullable=False)
    description = db.Column(db.Text, nullable=False)
    status = db.Column(db.String(20), default="Open")
    created_at = db.Column(db.DateTime, default=lambda: datetime.now(timezone.utc))
    user_id = db.Column(db.Integer, db.ForeignKey("users.id"), nullable=False)
```

Listing 4.1: SQLAlchemy models (app.py)

On start-up, `init_db()` creates the tables and seeds a default administrator and sample student accounts (with hashed passwords) and one sample complaint, so that a freshly deployed stack is immediately demonstrable.

4.2 Authentication and Security

Registration validates that both fields are present, rejects duplicate usernames (case-insensitive), and stores `generate_password_hash(password)`. Login looks the user up case-insensitively and verifies the hash with `check_password_hash`; on success the user id, username and role are stored in the session and the user is redirected by role.

An important hardening step concerns the registration form. An earlier version read the role from the submitted form, which would have let anyone register as an administrator by editing the request. The route now fixes the role on the server, and automated test 11 guards against regression:

```
@app.route("/register", methods=["GET", "POST"])
def register():
    if request.method == "POST":
        username = request.form.get("username", "").strip()
        password = request.form.get("password", "").strip()
        role = "student"   # never trust a role sent by the client
        ...
        new_user = User(username=username,
                        password=generate_password_hash(password), role=role)
```

Listing 4.2: Server-side role assignment during registration

4.3 Complaint Workflow

The `/complaints` route requires a session. A POST creates a `Complaint` bound to the logged-in user, commits it, increments the Prometheus counter `campusfix_complaints_total` and flashes a confirmation; a GET lists only that user's complaints in descending order of creation time. The `/admin`

route is restricted to the `admin` role and updates a complaint's status only if the submitted value is one of the three permitted states.

```
if request.method == "POST":
    complaint_id = request.form.get("complaint_id")
    new_status = request.form.get("status")
    complaint = db.session.get(Complaint, int(complaint_id)) if complaint_id else None
    if complaint and new_status in ["Open", "In Progress", "Resolved"]:
        complaint.status = new_status
        db.session.commit()
        flash(f"Complaint #{complaint.id} status updated to {new_status}.", "success")
```

Listing 4.3: Status update in the admin dashboard

4.4 DevOps Endpoints

Three endpoints were added specifically for operations. **/health** executes `SELECT 1` against the database and returns JSON with `status: healthy` and `database: connected` (HTTP 200), or HTTP 500 with the error if the database is unreachable. It is used by the Docker `HEALTHCHECK`, by Compose health checks and by the Jenkins smoke test. **/metrics** is provided by `prometheus-flask-exporter` and publishes request counts, latency histograms and the custom complaint counter. **/api/stats** returns the number of complaints by status.

```
@app.route("/health")
def health():
    try:
        db.session.execute(db.text("SELECT 1"))
        return jsonify({"status": "healthy", "service": "CampusFix",
                        "database": "connected",
                        "timestamp": datetime.now(timezone.utc).isoformat()}), 200
    except Exception as e:
        return jsonify({"status": "unhealthy", "error": str(e)}), 500
```

Listing 4.4: Health-check endpoint

4.5 User Interface

The interface consists of six Jinja2 templates extending a shared `base.html` layout: the home page, login, registration, the student's complaint page (submission form plus table of own complaints with status badges) and the administrator dashboard (all complaints with a status selector). Flash messages give immediate feedback for success and error cases.

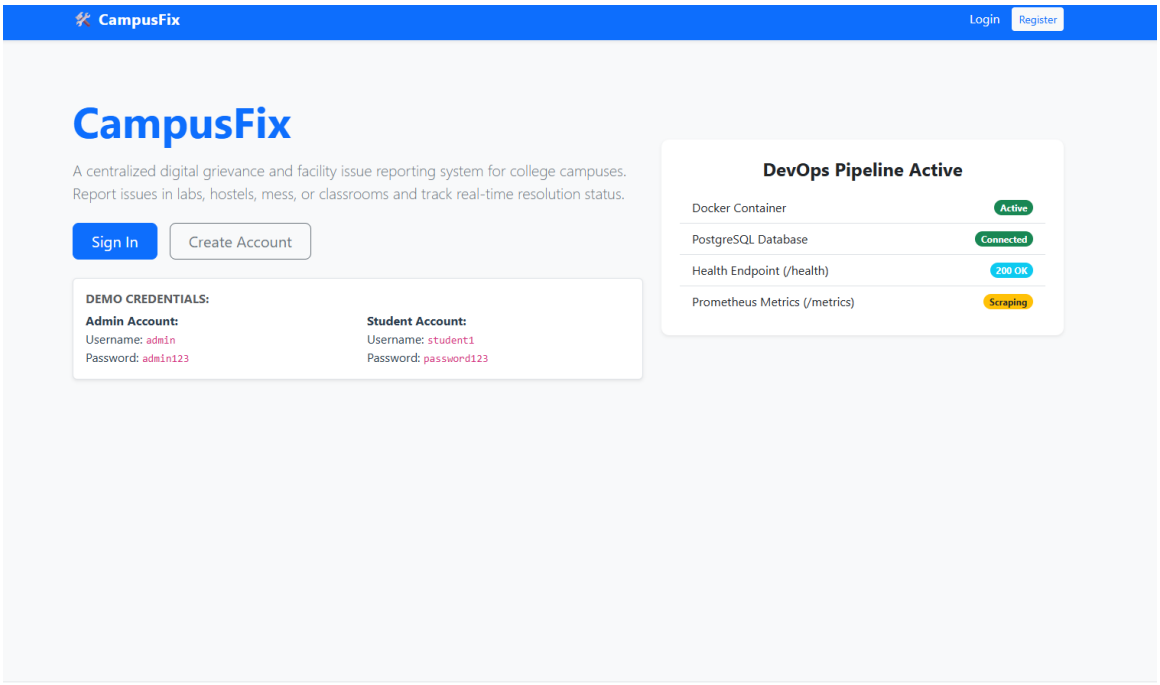


Figure 4.1: Home page

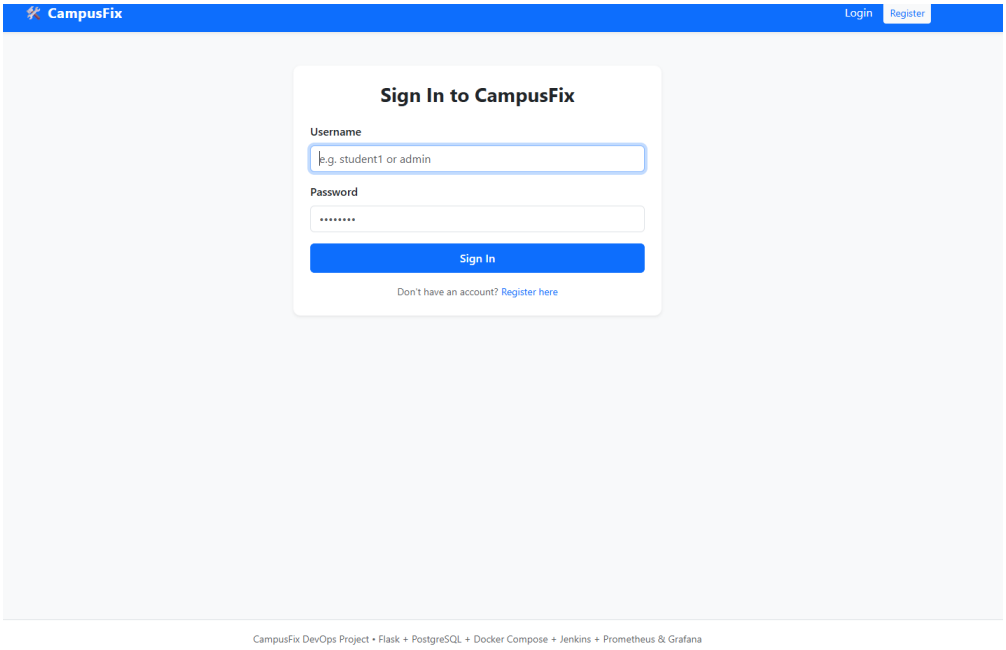


Figure 4.2: Login page

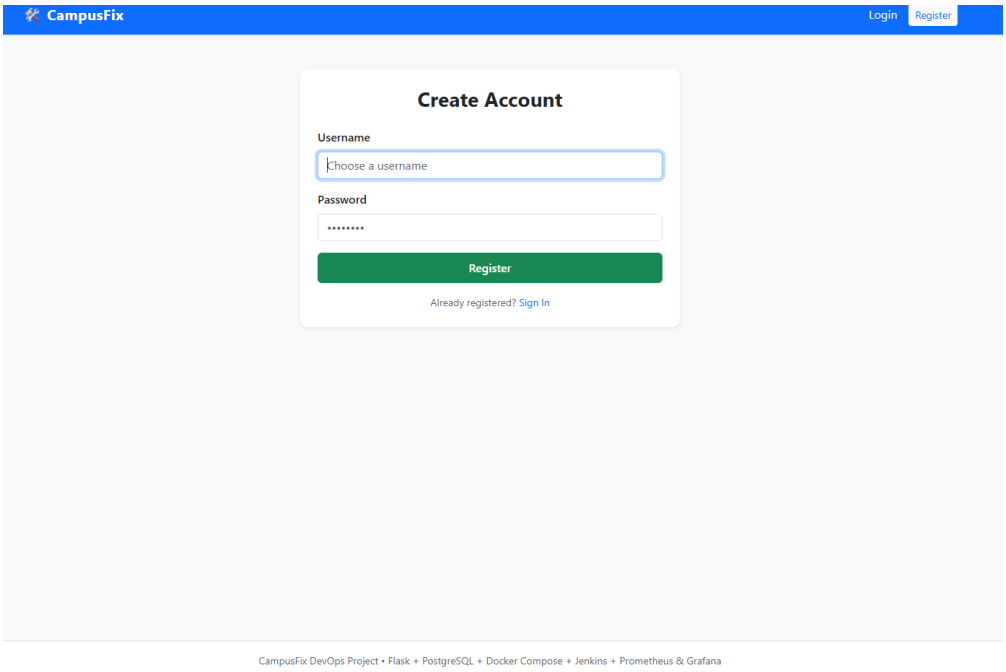


Figure 4.3: Registration page

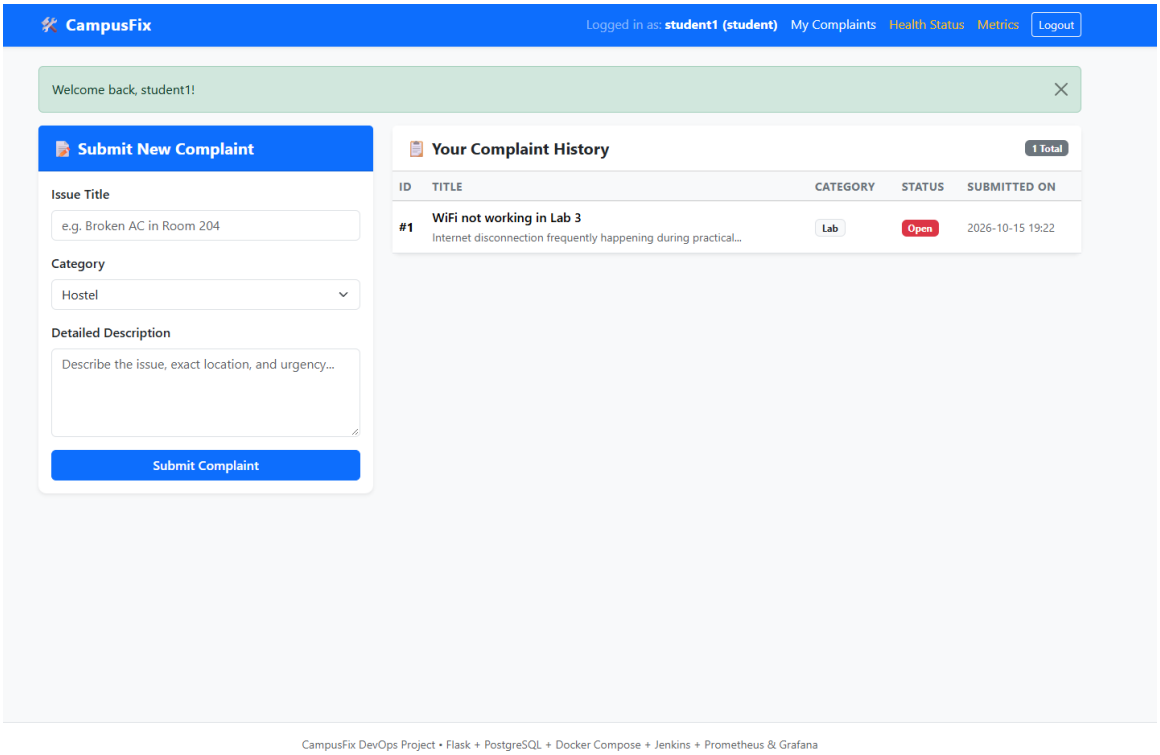


Figure 4.4: Student grievance portal: submission form and list of own complaints

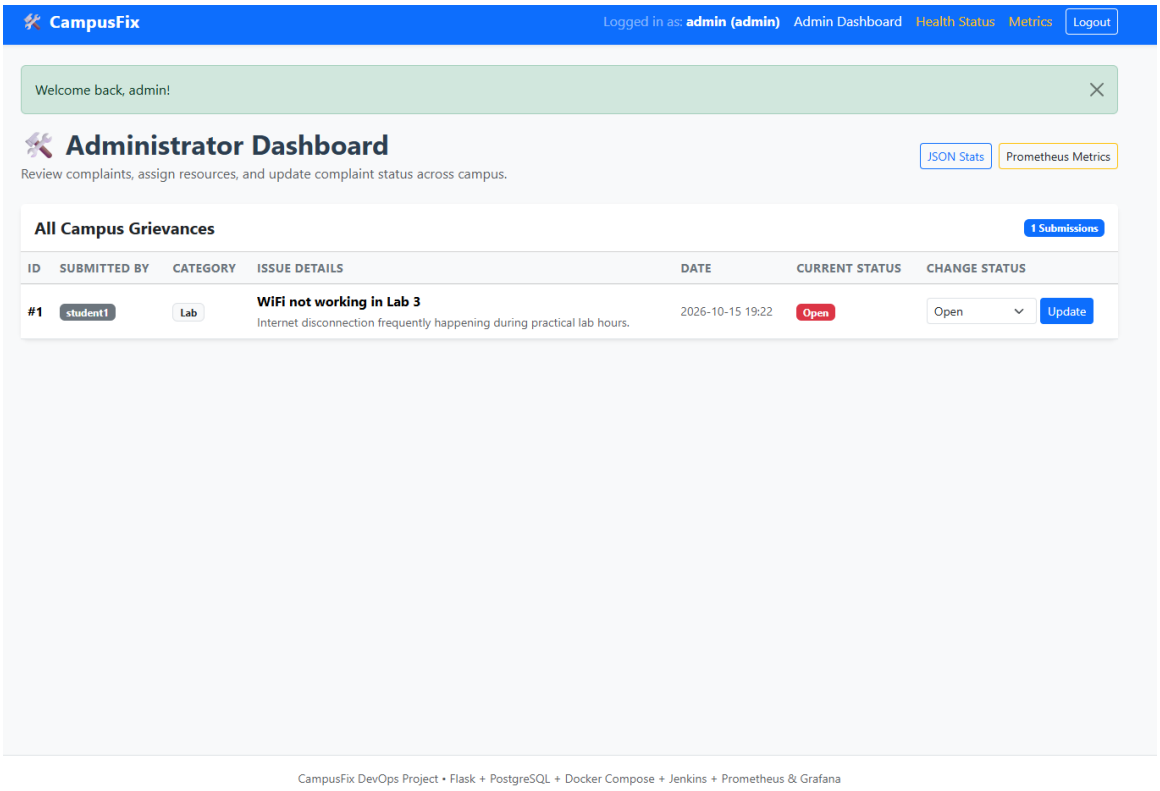


Figure 4.5: Administrator dashboard with status update controls

5. DevOps Implementation

This chapter describes how each laboratory experiment is applied in CampusFix. Figure 5.1 summarises the delivery pipeline that links them: a change is committed to Git, Jenkins checks it out and runs the tests inside a container, a production image is built, the stack is deployed with Docker Compose, an automated smoke test verifies `/health`, and Prometheus and Grafana then observe the running system.

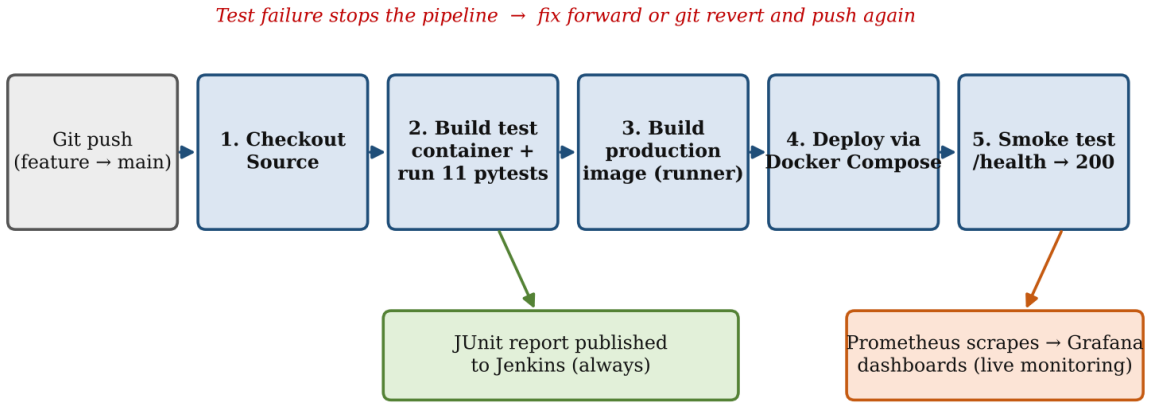


Figure 5.1: Delivery pipeline from commit to monitoring

5.1 Version Control with Git and GitHub (Experiment 1)

The source code is hosted in the GitHub repository **Moheet10/campusfix**. Development follows a branch-per-change model: a feature or fix is made on its own branch (for example `feature/auth-enhancement` or `fix/broken-logic`), pushed to GitHub and merged into `main` with a non-fast-forward merge commit so that the history of each change remains grouped and traceable. Commit messages follow a conventional prefix style such as `feat(core): ...` and `fix(auth): ...`.

The workflow was demonstrated end to end with a deliberate failure. A commit that breaks a test was pushed to `main`; the Jenkins pipeline failed in Stage 2 and refused to deploy. The faulty commit was then undone with `git revert`, which creates a new commit that reverses the change without rewriting history, and the next build returned to green.

```
# Rollback without rewriting history
git revert HEAD --no-edit
git push origin main
```

Listing 5.1: Rollback with git revert

Build	Git action	Jenkins result	Outcome
1	Initial commit on `main`	SUCCESS	All tests passed; stack deployed on port 5000
2	Merge of `feature/auth-enhancement`	SUCCESS	Non-fast-forward merge verified and redeployed
3	Commit with a deliberately failing assertion	FAILED	Failure caught by tests in Stage 2; deployment aborted
4	`git revert HEAD`	SUCCESS	Rollback restored a passing, deployable build

Table 5.1: Git workflow and Jenkins outcomes

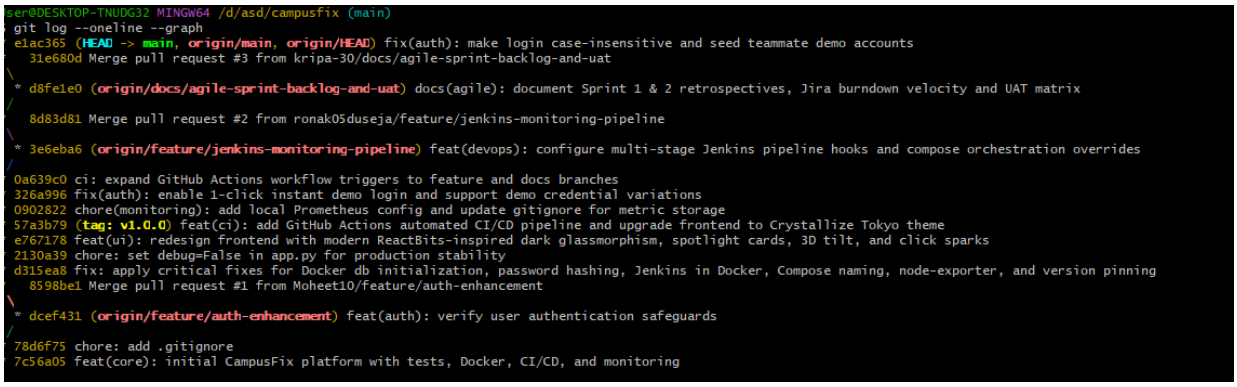


Figure 5.2: Git commit history showing feature branches and pull-request merge commits

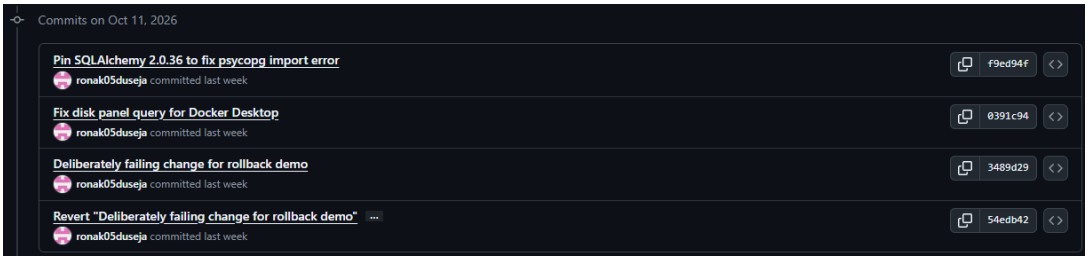


Figure 5.3: Git log showing the deliberate failing commit and the git revert commit

5.2 Containerisation with Docker (Experiment 2)

The Dockerfile is a **three-stage build**, which keeps the production image small and lets the same file serve both testing and release:

- **builder** – starts from `python:3.11-slim`, installs build tools and `libpq-dev`, and installs the Python dependencies into an isolated prefix (`/install`).
- **tester** – copies the installed packages and the source and runs `pytest -v --junitxml=junit-report.xml`. Jenkins builds this target to execute the test suite.
- **runner** – the production image. It installs only the runtime library `libpq5` and `curl`, creates the unprivileged user `appuser` (UID 10001), copies the packages and code, switches to that user, declares a `HEALTHCHECK` against `/health` and starts the application with Gunicorn (1 worker, 4 threads).

```
FROM python:3.11-slim AS runner
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends libpq5 curl \
    && rm -rf /var/lib/apt/lists/*
RUN groupadd -g 10001 appgroup && useradd -u 10001 -g appgroup -s /bin/sh -m appuser
COPY --from=builder /install /usr/local
COPY . .
RUN chown -R appuser:appgroup /app
USER appuser
EXPOSE 5000
HEALTHCHECK --interval=15s --timeout=5s --start-period=10s --retries=3 \
    CMD curl -f http://localhost:5000/health || exit 1
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "1", "--threads", "4",
"app:app"]
```

Listing 5.2: Production stage of the Dockerfile

Running as a non-root user limits the damage if the application is ever compromised, and the single Gunicorn worker avoids two processes trying to create tables and seed accounts at the same time during start-up.



Figure 5.4: docker build output for the multi-stage Dockerfile

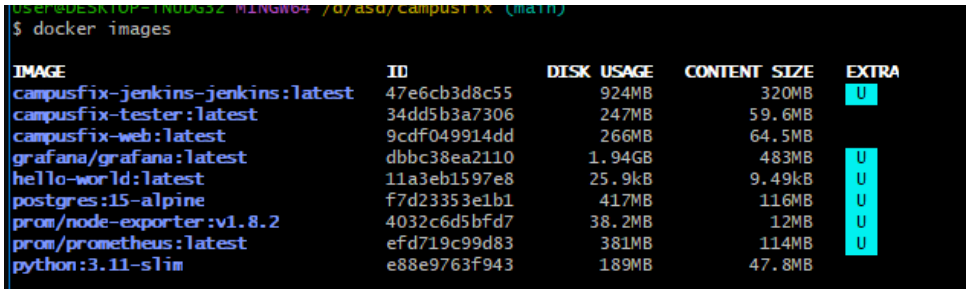


Figure 5.5: docker images showing the size of the production image

5.3 Multi-Container Orchestration with Docker Compose (Experiment 3)

A single `docker-compose.yml` (project name `campusfix`) defines five services on the bridge network `campusfix-network`:

Service	Image / build	Port	Purpose
db	postgres:15-alpine	5432	PostgreSQL database; volume `postgres_data`; health check `pg_isready`
web	Built from `./Dockerfile`, target `runner`	5000	Flask + Gunicorn application; waits for a healthy database
node-exporter	prom/node-exporter:v1.8.2	9100	Host CPU, memory and disk metrics
prometheus	prom/prometheus	9090	Metrics collector configured by `monitoring/prometheus.yml`
grafana	grafana/grafana	3000	Dashboards; volume `grafana_data`

Table 5.2: Services in the Compose stack

Two refinements make start-up dependable. First, **health-gated start-up**: the `db` service defines a `pg_isready` health check and `web` declares `depends_on: db: condition: service_healthy`, so the application never starts before the database accepts connections. Second, **persistence**: the named volumes `postgres_data` and `grafana_data` keep user accounts, complaints and Grafana settings across restarts. The database URL and secret key are passed to `web` as environment variables.

```
  web:
    build:
      context: .
      target: runner
    depends_on:
      db:
        condition: service_healthy
    environment:
      DATABASE_URL: postgresql://campusfix:***@db:5432/campusfix_db
    ports:
      - "5000:5000"
```

Listing 5.3: Excerpt of docker-compose.yml (web service)



Figure 5.6: docker compose ps showing all services running and healthy

5.4 Continuous Integration with Jenkins (Experiment 4)

Jenkins runs a declarative pipeline defined in the `Jenkinsfile` at the repository root. The pipeline has five stages (Table 5.3). The test stage builds the `tester` image target and runs pytest in a container; the JUnit XML is copied out of the container and published in a `post { always { ... } }` block, so results appear on the Jenkins dashboard even when tests fail. If any test fails, the pipeline stops and no later stage runs.

Stage	Steps	Purpose
1. Checkout Source	`checkout scm`	Fetch the latest code from GitHub
2. Build Test Container & Execute Pytests	`docker build --target tester`; run pytest with `--junitxml`; `docker cp` the report; publish with `junit`	Run all 11 tests in a clean container and publish the report
3. Build Production Multi-Stage Image	`docker build --target runner -t campusfix-web:latest .`	Create the hardened production image
4. Deploy via Docker Compose	`docker compose up -d --build db web`	Deploy the application and database
5. Automated Smoke Test (/health)	Up to five attempts of `curl` against `/health` inside `campusfix-web`, 3 s apart	Confirm the deployment returns HTTP 200

Table 5.3: Jenkins pipeline stages

```
stage('2. Build Test Container & Execute Pytests') {
    steps {
        sh 'docker build --target tester -t campusfix-tester .'
        sh '''
            docker rm -f cf-test >/dev/null 2>&1 || true
            set +e
            docker run --name cf-test campusfix-tester pytest -v --junitxml=/tmp/junit-
report.xml
            rc=$?
            docker cp cf-test:/tmp/junit-report.xml junit-report.xml
            docker rm -f cf-test >/dev/null 2>&1
            exit $rc
        '''
    }
    post { always { junit 'junit-report.xml' } }
}
```

Listing 5.4: Test stage of the Jenkinsfile

Capturing the exit code, copying the report and only then exiting with the original code is deliberate: it guarantees the report is archived for both passing and failing runs while still failing the build when a test fails.

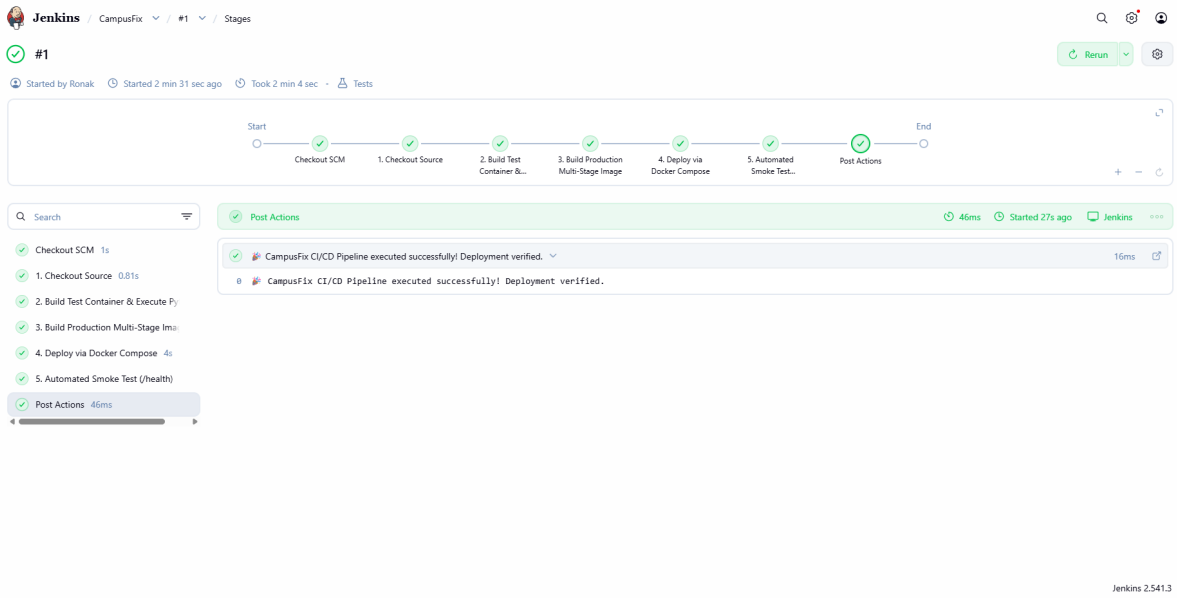


Figure 5.7: Jenkins pipeline stage view with all five stages green

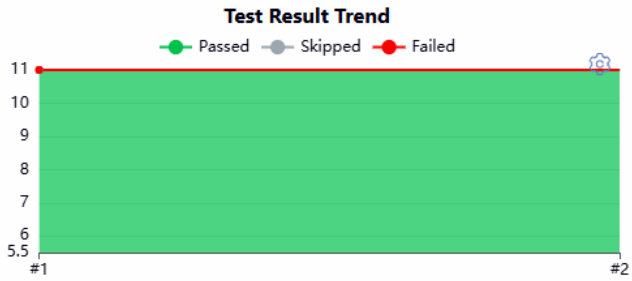


Figure 5.8: Jenkins JUnit test result trend

5.5 Automated Deployment and Smoke Testing (Experiment 5)

Stages 3 to 5 form the delivery half of the pipeline. After the production image is built, Docker Compose deploys the `db` and `web` services. The smoke test then runs `curl` inside the `campusfix-web` container against `http://localhost:5000/health` and succeeds only on HTTP 200, retrying up to five times to allow for start-up time. A deployment is therefore reported successful only if the application is up **and** can reach its database, since `/health` executes a database query.

```
for i in 1 2 3 4 5; do
    STATUS=$(docker exec campusfix-web curl -s -o /dev/null -w "%{http_code}"
http://localhost:5000/health || true)
    if [ "$STATUS" = "200" ]; then
        echo "Application healthy and smoke test passed with HTTP 200!"
        exit 0
    fi
    sleep 3
done
echo "Smoke test failed: /health did not return HTTP 200"; exit 1
```

Listing 5.5: Smoke-test stage

Recovery from a bad release is done through Git: the faulty commit is reverted (Section 5.1) and the pipeline redeploys the last known-good behaviour. A secondary GitHub Actions workflow (`.github/workflows/ci.yml`) also runs the pytest suite, archives the JUnit report and builds the `tester` and `runner` image targets on pushes and pull requests, giving an independent check outside Jenkins.

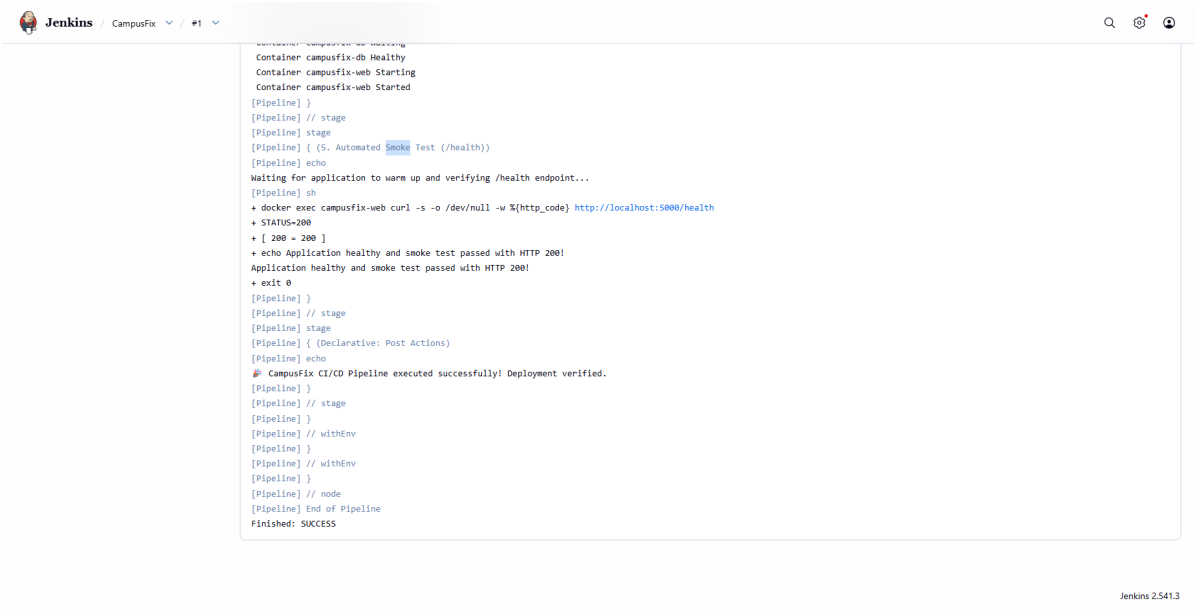


Figure 5.9: Jenkins console output of the smoke test stage showing HTTP 200

5.6 Monitoring with Prometheus and Grafana (Experiment 7)

Observability is built into the application and the Compose stack. The Flask app is instrumented with `prometheus-flask-exporter`, which publishes standard request metrics at `/metrics`, plus an application-specific counter `campusfix_complaints_total`. Prometheus is configured with three scrape jobs at a five-second interval:

```
global:
  scrape_interval: 5s
scrape_configs:
  - job_name: 'prometheus'
    static_configs: [{ targets: ['localhost:9090'] }]
  - job_name: 'campusfix-web'
    metrics_path: '/metrics'
    static_configs: [{ targets: ['web:5000'] }]
  - job_name: 'node'
    static_configs: [{ targets: ['node-exporter:9100'] }]
```

Listing 5.6: monitoring/prometheus.yml

Grafana loads the pre-built dashboard `monitoring/grafana-dashboard.json`, “CampusFix DevOps Monitoring Dashboard”, which contains seven panels:

Panel	Type	Query / metric
CampusFix Web Health Status	Stat	`up{job="campusfix-web"}`
Total Complaints Submitted	Stat	`campusfix_complaints_total`
HTTP Request Rate (req/sec)	Time series	`sum(rate(flask_http_request_total[1m]))`
Request Duration Latency (p95)	Time series	`histogram_quantile(0.95, ... flask_http_request_duration_seconds_bucket ...)`
CPU Usage (%)	Gauge	`100 - avg(rate(node_cpu_seconds_total{mode="idle"}[1m])) * 100`
Memory Usage (%)	Gauge	`(1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes) * 100`
Disk Usage (%)	Gauge	Root filesystem used percentage from node-exporter

Table 5.4: Grafana dashboard panels

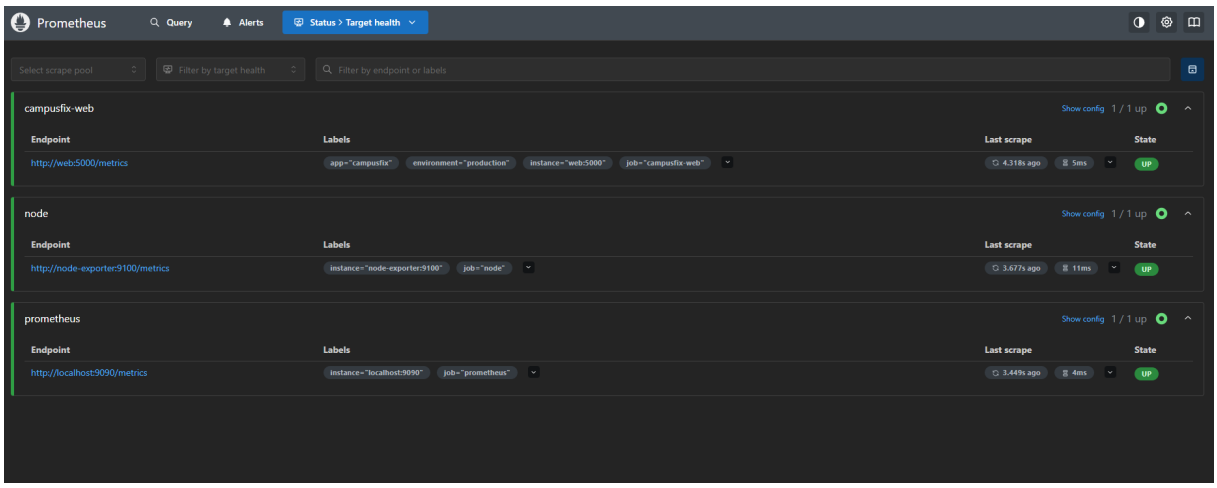


Figure 5.10: Prometheus targets page showing campusfix-web and node targets UP



Figure 5.11: Grafana dashboard showing live request rate, p95 latency, complaint counter and host gauges

6. Testing

6.1 Test Strategy

Testing combines automated unit and integration tests, run on every pipeline execution, with manual user-acceptance scenarios and automated smoke tests of the deployed stack. The pytest suite uses Flask's test client and an `autouse` fixture that, before each test, creates the tables, clears all rows and seeds one administrator, one student and one complaint, and cleans up again afterwards, so each test starts from a known state and tests cannot affect one another. When `DATABASE_URL` is not set, the application falls back to SQLite, so the suite runs in the lightweight `tester` container without a separate database.

6.2 Automated Test Cases

#	Test function	Scenario	Expected result
1	test_home_page	GET `/`	HTTP 200 and page contains "CampusFix"
2	test_health_check	GET `/health`	HTTP 200, status healthy, database connected
3	test_metrics_endpoint	GET `/metrics`	HTTP 200 and Prometheus metric names present
4	test_api_stats	GET `/api/stats`	HTTP 200; total ≥ 1 and open ≥ 1
5	test_user_registration	POST `/register` with a new username	"Registration successful" message
6	test_duplicate_registration_fails	POST `/register` with an existing username	"Username already exists" message
7	test_login_success	POST `/login` with valid credentials	"Welcome back" message
8	test_login_invalid_credentials	POST `/login` with a wrong password	"Invalid credentials" message
9	test_complaint_submission	Student logs in and posts a complaint	Success message; complaint visible in the list
10	test_admin_access_control_and_status_update	Admin opens `/admin` and sets a complaint to Resolved	Dashboard shown; "status updated to Resolved"
11	test_register_role_escalation_prevented	POST `/register` with `role=admin`	Created user has role `student`

Table 6.1: Automated pytest test cases

6.3 Test Results

The suite was executed with `pytest -v --junitxml=junit-report.xml`. The generated JUnit report records **11 tests, 0 failures, 0 errors and 0 skipped**, with a total run time of about 3.9 seconds. In the pipeline the same report is published to Jenkins, which plots the pass/fail trend across builds.

```
$ pytest -v
tests/test_app.py::test_home_page PASSED
tests/test_app.py::test_health_check PASSED
...
tests/test_app.py::test_register_role_escalation_prevented PASSED
============ 11 passed in 3.93s ============
```

Listing 6.1: Summary of pytest output

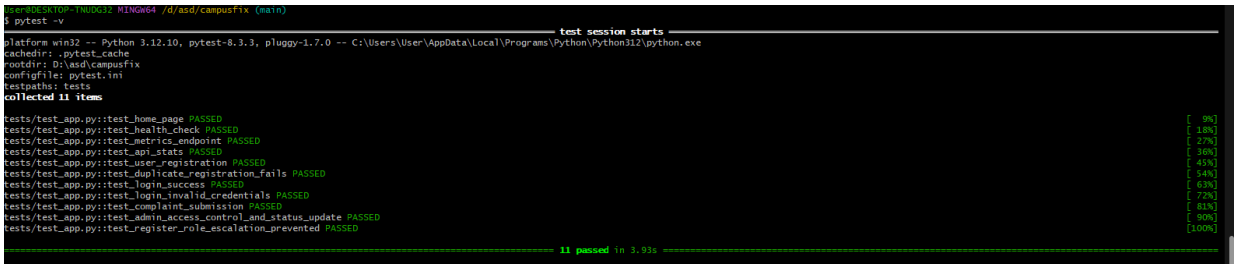


Figure 6.1: Terminal running pytest -v showing 11 of 11 tests passed

6.4 User Acceptance Testing

ID	Persona	Scenario	Expected outcome	Status
UAT-01	Student	Register with a unique username and password	Account created; redirected to login with success alert	PASS
UAT-02	Student	Submit a complaint (Hostel / Wi-Fi breakdown)	Ticket stored; complaint counter increments	PASS
UAT-03	Student	View the filed complaints table	Ticket shown with status badge Open and timestamp	PASS
UAT-04	Admin	Sign in with administrator credentials	Directed to the `/admin` console	PASS
UAT-05	Admin	Change a ticket from Open to Resolved	Status changes; confirmation banner shown	PASS
UAT-06	Student	Try to register as admin by editing the form	Server assigns role student	PASS
UAT-07	DevOps	Query the `/health` endpoint	HTTP 200 with `{"status": "healthy"}`	PASS
UAT-08	DevOps	Check Prometheus targets for the app	Target state UP	PASS

Table 6.2: User acceptance test matrix

6.5 Smoke Test of the Deployed Stack

After deployment, the pipeline calls `/health` and requires HTTP 200 (Section 5.5). Independently, the Prometheus targets page confirms that the application and node-exporter targets are scraped successfully, and the Docker health status of `campusfix-web` and `campusfix-db` shows `healthy`.

7. Results and Discussion

7.1 Summary of Results

Aspect	Result
Functionality	Registration, login, complaint submission, own-complaint listing, admin status updates and access control all work as specified.
Automated tests	11 of 11 pytest tests pass (0 failures, 0 errors).
Pipeline	Five-stage Jenkins pipeline: checkout, tests in container with JUnit report, production image build, Compose deployment, `/health` smoke test.
Containers	Three-stage Dockerfile; production image runs as non-root user with a health check. Production image size: 266 MB on disk (64.5 MB compressed content size).
Orchestration	Five-service stack starts with one command; web waits for a healthy database; data persists in named volumes.
Monitoring	Prometheus scrapes the app and node-exporter every 5 s; Grafana dashboard with 7 panels shows health, throughput, latency, complaints and host resources.
Agile	Two sprints, eight stories and five epics tracked in Jira (37 story points planned).

Table 7.1: Summary of results

7.2 Discussion

The project shows how the individual experiments reinforce one another. The tests created in the application phase become the quality gate of the Jenkins pipeline; the Dockerfile's dedicated `tester` stage lets the same repository be tested and shipped without extra tooling; the `/health` endpoint serves three consumers at once (Docker, Compose and the smoke test); and the metrics endpoint feeds the monitoring stack. Because every step is scripted, a clean machine with Docker can reproduce the whole environment in minutes. Placing a pass/fail gate before deployment means a broken commit never reaches the running system, and recovery is a single, auditable `git revert`.

7.3 Challenges and Resolutions

Table 7.2 lists the main problems met during development and how each was resolved.

Challenge (Table 7.2)	Resolution
The application could crash at start-up if PostgreSQL was not yet accepting connections.	Added a `pg_isready` health check to the database service and `depends_on: condition: service_healthy` on the web service.
Multiple Gunicorn workers could try to create tables and seed accounts simultaneously, causing database-initialisation errors.	Configured Gunicorn with one worker and four threads.
The registration form accepted a client-supplied role, allowing privilege escalation.	Role is now fixed to `student` on the server; regression test 11 added.
Test report was lost when a test failed inside the container.	The pipeline captures the exit code, copies the JUnit XML out of the container, then exits with the original code; the report is published in a `post always` block.
Login could fail because of differences in username capitalisation.	Username lookup was made case-insensitive for both registration and login.

8. Conclusion and Future Scope

8.1 Conclusion

CampusFix delivers a working campus grievance platform and, more importantly for this course, a complete Agile and DevOps delivery chain around it. Students can register and file categorised complaints, administrators can drive them to resolution, and the whole system is version-controlled, automatically tested, containerised, deployed through a Jenkins pipeline, smoke-tested and monitored. All seven laboratory experiments were applied to one realistic product rather than to isolated exercises, which made their dependencies clear: tests gate the pipeline, containers make deployments reproducible, health endpoints enable automation and metrics make runtime behaviour visible.

8.2 Future Scope

- Add e-mail or push notifications when a complaint changes status, and complaint assignment to named staff or departments.
- Support photo attachments and priority levels, with escalation rules for complaints that remain open for too long.
- Move secrets (database password, secret key) into the Jenkins credentials store or a secrets manager, and add HTTPS through a reverse proxy such as Nginx.
- Add Prometheus alert rules and Alertmanager notifications, for example when `/health` fails or p95 latency rises.
- Extend the pipeline with static analysis, dependency and image vulnerability scanning, and automated rollback on a failed smoke test.
- Deploy to a cloud VM or Kubernetes cluster and introduce database migrations (for example Alembic) for schema evolution.
- Add analytics for administrators, such as average resolution time per category and trends by hostel or lab.

References

1. Pallets Projects, “Flask Documentation (3.0.x).” https://flask.palletsprojects.com/
2. SQLAlchemy Project, “Flask-SQLAlchemy Documentation.” https://flask-sqlalchemy.palletsprojects.com/
3. Pallets Projects, “Werkzeug Security Utilities.” https://werkzeug.palletsprojects.com/
4. Gunicorn, “Gunicorn Documentation.” https://docs.gunicorn.org/
5. pytest Development Team, “pytest Documentation.” https://docs.pytest.org/
6. Docker Inc., “Dockerfile reference” and “Multi-stage builds.” https://docs.docker.com/
7. Docker Inc., “Docker Compose overview.” https://docs.docker.com/compose/
8. Jenkins Project, “Pipeline Syntax (Declarative Pipeline).” https://www.jenkins.io/doc/book/pipeline/syntax/
9. Prometheus Authors, “Prometheus Documentation.” https://prometheus.io/docs/
10. Grafana Labs, “Grafana Documentation.” https://grafana.com/docs/
11. Atlassian, “Jira Software: Scrum.” https://www.atlassian.com/software/jira/guides
12. Pro Git (S. Chacon, B. Straub), “git revert.” https://git-scm.com/book/
13. CampusFix source repository. https://github.com/Moheet10/campusfix

Appendix A: Running the Project

A.1 Run the tests locally

```
python -m venv .venv
source .venv/bin/activate     # Windows: .\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
pytest -v --junitxml=junit-report.xml
```

A.2 Start the full stack

```
docker compose up -d --build
docker compose ps
docker compose logs -f web
```

A.3 Service endpoints

Service	URL	Notes
CampusFix web app	http://localhost:5000	Student: `student1` / `password123`; Admin: `admin` / `admin123`
Health check	http://localhost:5000/health	JSON health status
Metrics	http://localhost:5000/metrics	Prometheus text format
Prometheus	http://localhost:9090	Check Status → Targets
Grafana	http://localhost:3000	Login `admin` / `admin`; import `monitoring/grafana-dashboard.json`

Table A.1: Service endpoints (demo credentials for the lab environment only)

A.4 Run the Jenkins pipeline

1. Start Jenkins (for example in Docker with access to the host Docker daemon) and install the Git, Pipeline and JUnit plugins.
2. Create a Pipeline job pointing at the GitHub repository with “Pipeline script from SCM” and the `Jenkinsfile` at the root.
3. Run the build and open the stage view and the test result trend.

Appendix B: Team Contributions

Member	Role	Core tasks	Experiments
Moheet Dialani	Application & Containers	Flask backend, authentication and security hardening, 11 pytest tests, Dockerfile, Docker Compose, `/health` and `/metrics` endpoints, repository, Jenkinsfile and monitoring configuration files	2, 3 (foundation for 1, 4, 5, 7)
Ronak Duseja	CI/CD & Monitoring	Jenkins setup and pipeline execution, automated deployment, git revert rollback demonstration, Prometheus and Grafana	1, 4, 5, 7
Kripa Masand	Agile & Presentation	Jira project, epics, stories, sprints and burndown, QA/acceptance test table, report assembly, viva	6

Table B.1: Contribution of each team member